

```
import pandas as pd
df = pd.read_csv("Messy_Employee_dataset.csv")
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] = df["Salary"].fillna(df["Salary"].median())
df = df.drop_duplicates()
print(df.head())
print(df.isnull().sum())
```

	Employee_ID	First_Name	Last_Name	Age	Department_Region	Status
0	EMP1000	Bob	Davis	25.0	DevOps-California	Active
1	EMP1001	Bob	Brown	30.0	Finance-Texas	Active
2	EMP1002	Alice	Jones	30.0	Admin-Nevada	Pending
3	EMP1003	Eva	Davis	25.0	Admin-Nevada	Inactive
4	EMP1004	Frank	Williams	25.0	Cloud Tech-Florida	Active

	Join_Date	Salary	Email	Phone
0	4/2/2021	59767.65	bob.davis@example.com	-1651623197
1	7/10/2020	65304.66	bob.brown@example.com	-1898471390
2	12/7/2023	88145.90	alice.jones@example.com	-5596363211
3	11/27/2021	69450.99	eva.davis@example.com	-3476490784
4	1/5/2022	109324.61	frank.williams@example.com	-1586734256

	Performance_Score	Remote_Work
0	Average	True
1	Excellent	True
2	Good	True
3	Good	True
4	Poor	False

```
Employee_ID      0
First_Name       0
Last_Name        0
Age              0
Department_Region 0
Status           0
Join_Date        0
Salary           0
Email            0
Phone           0
Performance_Score 0
Remote_Work      0
dtype: int64
```

```
# mean
marks = [45,50,55,60,90]
mean = sum(marks) / len(marks)
print("Mean:",mean)
```

Mean: 60.0

```
#median
import statistics
marks = [45,50,55,60,90]
median = statistics.median(marks)
print("Median",median)
```

Median 55

```
# mode
import statistics
data = [10,20,20,30,40]
mode = statistics.mode(data)
print("Mode:",mode)
```

Mode: 20

```
## variance code
import numpy as np
data = [10,12,14,16,18]
print(np.mean(data))
variance = np.var(data)
print("Variance:",variance)
```

14.0

Variance: 8.0

```
import pandas as pd
df=pd.DataFrame({
    'Marks': [45,50,55,60,90]
})
print("Mean:")
print(df["Marks"].mean())
print("\nMedian:")
print(df['Marks'].median())
print("\nMode:")
print(df['Marks'].mode())
```

Mean:
60.0

Median:
55.0

Mode:

```
0    45
1    50
2    55
3    60
4    90
Name: Marks, dtype: int64
```

```
import numpy as np
data = [10,12,14,16,18]
variance = np.var(data)
std=np.std(data)
print("Variance:",variance)
print("std:",std)
```

```
Variance: 8.0
std: 2.8284271247461903
```

```
import numpy as np
data=[5,10,15,20,25,30,35,40]
Q1 = np.percentile(data,25)
Q3 = np.percentile(data,75)
IQR = Q3 - Q1
print("Q1:",Q1)
print("Q3:",Q3)
print("IQR:",IQR)
```

```
Q1: 13.75
Q3: 31.25
IQR: 17.5
```

#code

```
import numpy as np
data=[5,10,15,20,25,30,35,1000]
Q1 = np.percentile(data,25)
Q3 = np.percentile(data,75)
IQR = Q3 - Q1
lower = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print(lower)
print(upper)
outliers = [x for x in data if x < lower or x > upper]
print("Outliers:",outliers)
```

```
-12.5
57.5
Outliers: [1000]
```

```
sample_space = {1,2,3,4,5,6}
event_A = {2,4,6}
print("Sample space(s):",sample_space)
```

```

print("Event A:", event_A)

# probability p(a)

P_A = len(event_A) / len(sample_space)
print("\nProbability of Event A")
print("P(A) =", P_A)

Sample space(s): {1, 2, 3, 4, 5, 6}
Event A: {2, 4, 6}

Probability of Event A
P(A) = 0.5

P_Disease=0.01
P_positive_given_Disease=0.95
P_positive=0.05
P_disease_given_positive=(
    P_positive_given_Disease*P_Disease)/P_positive
print(P_disease_given_positive)

0.18999999999999997

# conditional probability
event_A = {2,4,6}
event_B = {4,5,6}
intersection = event_A.intersection(event_B)
P_A_give_B = len(intersection) / len(event_B)
print("\nEvent B:", event_B)
print("A intersection b:", intersection)
print("P(A|B) =", P_A_give_B)

Event B: {4, 5, 6}
A intersection b: {4, 6}
P(A|B) = 0.6666666666666666

#Bayes' theorem
'''
example:
    Disease prevalence=1%
    test sensitivity=99%
    positive test probability=5%
'''
P_Disease=0.01
P_positive_given_Disease=0.95
P_positive=0.05
P_disease_given_positive=(
    P_positive_given_Disease*P_Disease)/P_positive
print("Bayes' theorem example")
print("P(disease) =", P_Disease)

```

```

print("P(Positive|Disease) =",P_positive_given_Disease)
print("P(Positive)=",P_positive)
print("P(Disease|Positive)=",
      round(P_disease_given_positive,4)
)
print("Percentage=",
      round(P_disease_given_positive*100,2),
      "%")

print(P_disease_given_positive)

```

Bayes'theorem example
 $P(\text{disease}) = 0.01$
 $P(\text{Positive}|\text{Disease}) = 0.95$
 $P(\text{Positive}) = 0.05$
 $P(\text{Disease}|\text{Positive}) = 0.19$
 Percentage= 19.0 %
 0.18999999999999997

```

import pandas as pd
df= pd. DataFrame({
    'Hours' : [1,2,3,4,5],
    'Marks' : [20,40,50,70,90]
})
corr = df['Hours'].corr(df['Marks'])
print("Pearson Correlation:",corr)
# close to +1
#strong positive relation

Pearson Correlation: 0.9948497511671097

```

```

import pandas as pd
df= pd. DataFrame({
    'Rank_Study' : [1,2,3,4,5],
    'Rank_Marks' : [2,1,3,5,4]
})
corr = df['Rank_Study'].corr(
    df['Rank_Marks'],
    method='spearman'
)

print("Spearman Correlation:",corr)

Spearman Correlation: 0.7999999999999999

```

#matrix correlation

```

import pandas as pd
df= pd. DataFrame({
    'Hours' : [1,2,3,4,5],
    'Marks' : [20,40,50,70,90],
    'Sleep' : [8,7,6,5,4]

```

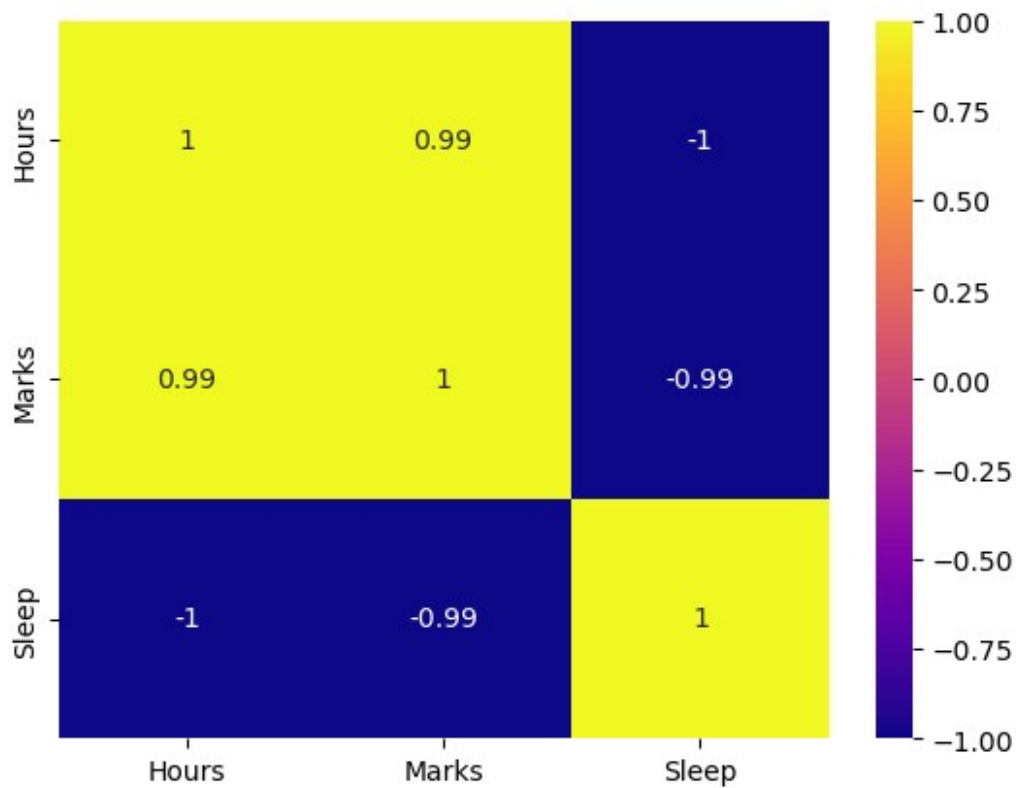
```
})
print(df.corr())
```

```

      Hours    Marks  Sleep
Hours  1.00000  0.99485 -1.00000
Marks  0.99485  1.00000 -0.99485
Sleep -1.00000 -0.99485  1.00000

```

```
##heatmap
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df= pd. DataFrame({
    'Hours' : [1,2,3,4,5],
    'Marks' : [20,40,50,70,90],
    'Sleep' : [8,7,6,5,4]
})
sns.heatmap(df.corr(),
annot=True,
cmap='plasma')
plt.show()
```



```
import numpy as np
import seaborn as sns
```

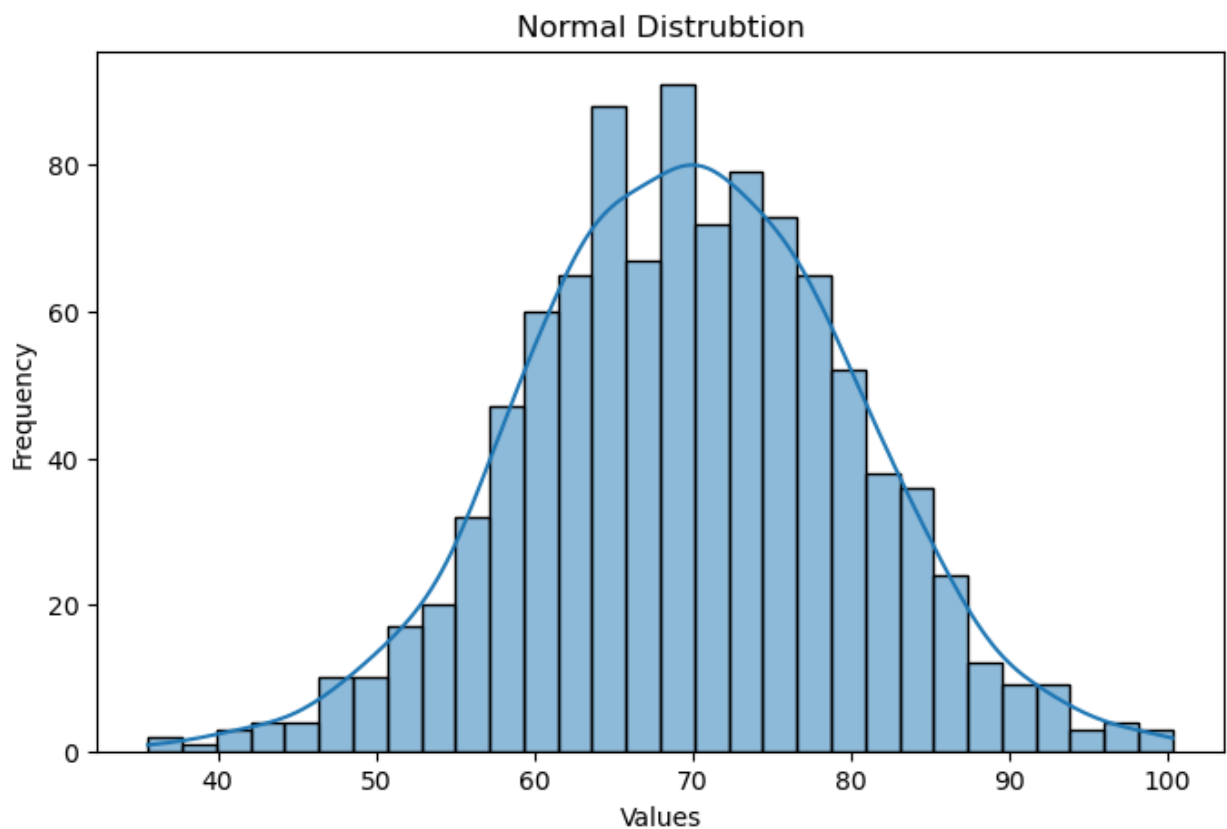
```

import matplotlib.pyplot as plt
#generate normally distributed data
data = np.random.normal(
    loc=70,      #mean
    scale=10,    #standard deviation
    size=1000   #number of values
)
#calculate mean and standard deviation
mean = np.mean(data)
std = np.std(data)
print("Mean:", round(mean,2))
print("Standard Deviation:", round(std,2))

plt.figure(figsize=(8,5))
sns.histplot(data,bins=30,kde=True)
plt.title("Normal Distrubtion")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()

```

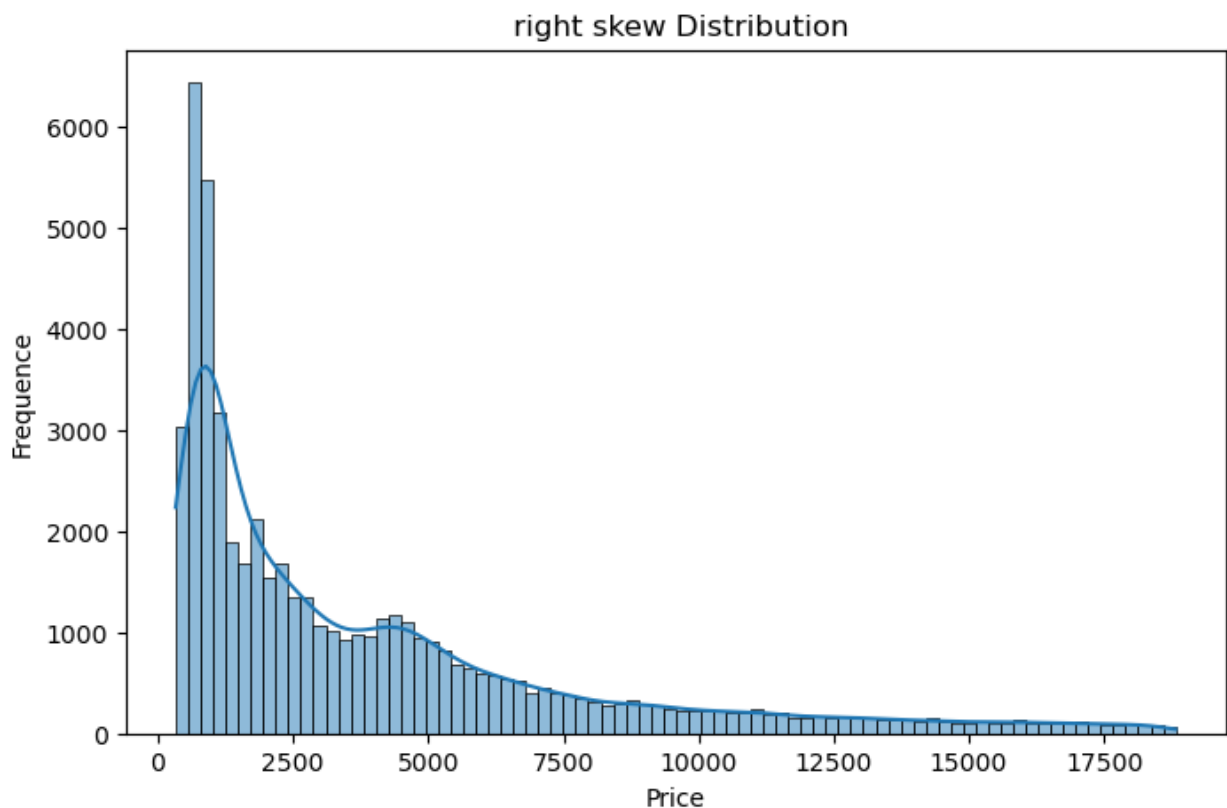
Mean: 69.76
Standard Deviation: 10.32



```

import seaborn as sns
import matplotlib.pyplot as plt
#load Dataset
df=sns.load_dataset("diamonds")
#histogram + KDE
plt.figure(figsize=(8,5))
sns.histplot(df["price"],kde=True)
plt.title(" right skew Distribution")
plt.xlabel("Price")
plt.ylabel("Frequence")
plt.show()

```



```

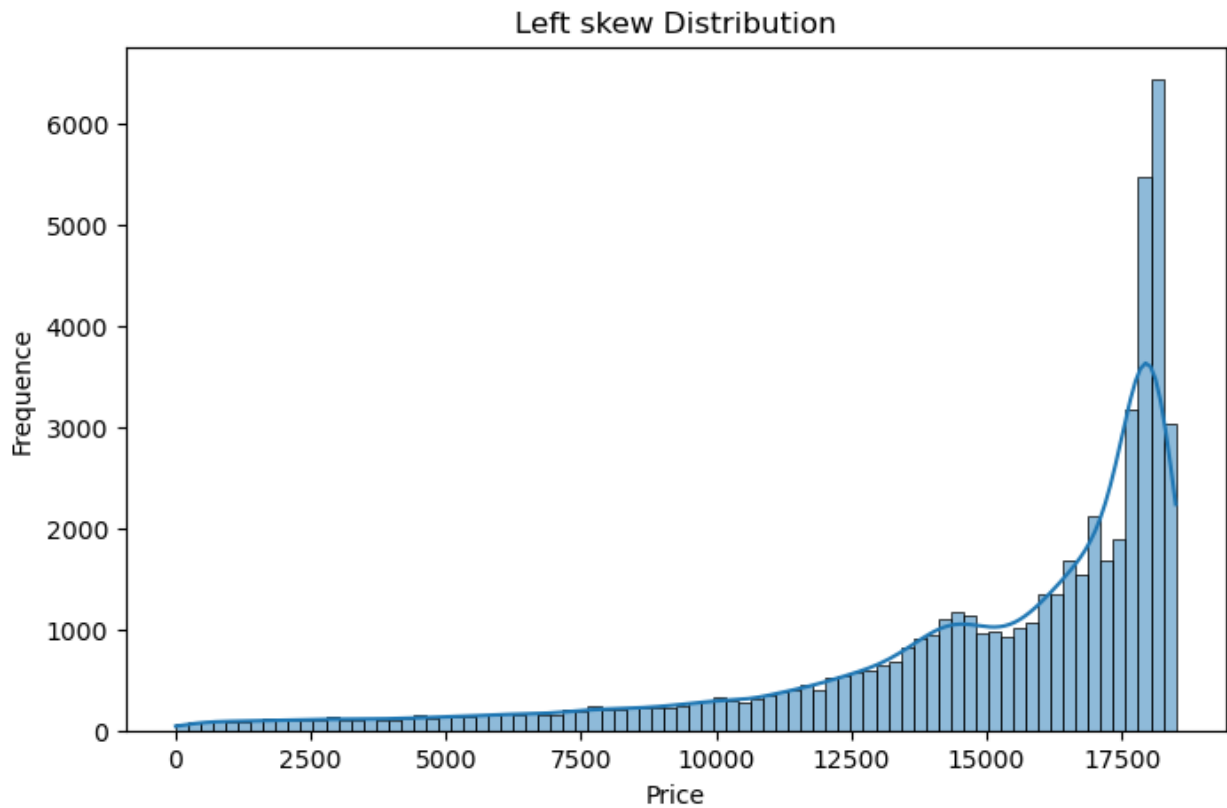
import seaborn as sns
import matplotlib.pyplot as plt
#load Dataset
df=sns.load_dataset("diamonds")

#create left-skewed data
left_skew = df["price"].max()-df["price"]
#histogram + KDE
plt.figure(figsize=(8,5))
sns.histplot(left_skew,kde=True)
plt.title("Left skew Distribution")
plt.xlabel("Price")

```



```
plt.ylabel("Frequency")
plt.show()
print("Skewness:", left_skew.skew())
```



Skewness: -1.6183952833835282

```
import numpy as np
scores = [60, 65, 70, 75, 80]
mean = np.mean(scores)
std = np.std(scores)
score = 80
z_score = (score - mean) / std
print("Mean:", mean)
print("Standard Deviation:", std)
print("Z-Score:", z_score)
```

Mean: 70.0
Standard Deviation: 7.0710678118654755
Z-Score: 1.414213562373095